```
"""
STEP 0: Install Required Libraries

This cell installs spaCy and downloads the English language model.
spaCy is used for Natural Language Processing tasks like tokenization,
POS tagging, and dependency parsing.
"""

# Install spaCy
!pip install spacy

# Download English model
!python -m spacy download en_core_web_sm
```

```
Requirement already satisfied: spacy in /usr/local/lib/python3.12/dist-packages (3.8.11)
Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.0.12)
Requirement already satisfied: spacy-loggers<2.0.0,>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.0.5)
Requirement already satisfied: murmurhash<1.1.0,>=0.28.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.0.15)
Requirement already satisfied: cymem<2.1.0,>=2.0.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.13)
Requirement already satisfied: preshed<3.1.0,>=3.0.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.0.12)
Requirement already satisfied: thinc<8.4.0,>=8.3.4 in /usr/local/lib/python3.12/dist-packages (from spacy) (8.3.10)
Requirement already satisfied: wasabi<1.2.0,>=0.9.1 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.1.3)
Requirement already satisfied: srsly<3.0.0,>=2.4.3 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.5.2)
Requirement already satisfied: catalogue<2.1.0,>=2.0.6 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.10)
Requirement already satisfied: weasel<0.5.0,>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (0.4.3)
Requirement already satisfied: typer-slim<1.0.0,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (0.24.0)
Requirement already satisfied: tqdm<5.0.0,>=4.38.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (4.67.3)
Requirement already satisfied: numpy>=1.19.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.2)
Requirement already satisfied: requests<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.32.4)
Requirement already satisfied: pydantic!=1.8,!=1.8.1,<3.0.0,>=1.7.4 in /usr/local/lib/python3.12/dist-packages (from spacy)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.1.6)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from spacy) (75.2.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (26.0)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!=1.8.1
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!=1.8.1,
Requirement already satisfied: typing-extensions>=4.14.1 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!=1.
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!=1.8
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0->spacy)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0->
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0->
Requirement already satisfied: blis<1.4.0,>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from thinc<8.4.0,>=8.3.4->spac
Requirement already satisfied: confection<1.0.0,>=0.0.1 in /usr/local/lib/python3.12/dist-packages (from thinc<8.4.0,>=8.3.4
Requirement already satisfied: typer>=0.24.0 in /usr/local/lib/python3.12/dist-packages (from typer-slim<1.0.0,>=0.3.0->spac
Requirement already satisfied: cloudpathlib<1.0.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from weasel<0.5.0,>=0.
Requirement already satisfied: smart-open<8.0.0,>=5.2.1 in /usr/local/lib/python3.12/dist-packages (from weasel<0.5.0,>=0.4.
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->spacy) (3.0.3)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart-open<8.0.0,>=5.2.1->weasel<0.5.0
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from typer>=0.24.0->typer-slim<1.0.0
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer>=0.24.0->typer-slim
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from typer>=0.24.0->typer-slim<1.0.0
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer>=0.24.0->typer-sl
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer>=0
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer>
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=12.3
Collecting en-core-web-sm==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/en_core_web_sm-3.8.0/en_core_web_sm-3.8.0-py3-none
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.8/12.8 MB 84.8 MB/s eta 0:00:00
✓ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in
order to load all the package's dependencies. You can do this by selecting the
'Restart kernel' or 'Restart runtime' option.
```

```
"""
STEP 1: Load spaCy Language Model

Language models contain statistical data required for NLP tasks such as:
- Tokenization
- POS tagging
- Dependency parsing

'en_core_web_sm' is a small English model suitable for basic NLP tasks.
"""

# Import spaCy
import spacy

# Load English model
nlp = spacy.load("en_core_web_sm")
```

```
# Verify model loading
print("spaCy model loaded successfully!")
```

```
spaCy model loaded successfully!
```

```
"""
STEP 2: Prepare Dataset

We define at least 15 sentences of different types:
- Simple
- Compound
- Complex
- Interrogative
- Passive voice

These sentences simulate real-world data such as news, social media, and technical text.
"""

sentences = [
    "The cat sits on the mat.",
    "She is reading a book.",
    "John and Mary went to the market.",
    "Although it was raining, we went outside.",
    "He completed the assignment and submitted it.",
    "The ball was thrown by the boy.",
    "Can you solve this problem?",
    "Artificial intelligence is transforming industries.",
    "The teacher explained the concept clearly.",
    "After the meeting ended, everyone left quickly.",
    "The car was repaired by the mechanic.",
    "She sings beautifully.",
    "Do you like programming?",
    "The scientist who discovered the formula won a prize.",
    "They are playing football in the ground."
]

print("Total sentences:", len(sentences))
```

```
Total sentences: 15
```

```
"""
STEP 3: Sentence Processing using spaCy

spaCy processes text using an NLP pipeline:
1. Tokenization → Splits text into words
2. POS Tagging → Assigns grammatical tags
3. Dependency Parsing → Identifies relationships between words
"""

# Process all sentences
docs = [nlp(sentence) for sentence in sentences]

print("Sentences processed successfully!")
```

```
Sentences processed successfully!
```

```
"""
STEP 4: Extract Dependency Relations

For each token in each sentence, we extract:
- Token text
- Lemma (base form)
- POS tag
- Dependency label
- Head word (parent in tree)

These help us understand grammatical structure.
"""

import pandas as pd

# Store all results
all_data = []

# Loop through sentences
for doc in docs:
    for token in doc:
        all_data.append([
            doc.text,
            token.text,
            token.lemma_,
```

```
            token.pos_,
            token.dep_,
            token.head.text
        ])

# Create DataFrame
df = pd.DataFrame(all_data, columns=[
    "Sentence", "Token", "Lemma", "POS", "Dependency", "Head"
])

# Display first 20 rows
df.head(20)
```

	Sentence	Token	Lemma	POS	Dependency	Head
0	The cat sits on the mat.	The	the	DET	det	cat
1	The cat sits on the mat.	cat	cat	NOUN	nsubj	sits
2	The cat sits on the mat.	sits	sit	VERB	ROOT	sits
3	The cat sits on the mat.	on	on	ADP	prep	sits
4	The cat sits on the mat.	the	the	DET	det	mat
5	The cat sits on the mat.	mat	mat	NOUN	pobj	on
6	The cat sits on the mat.	.	.	PUNCT	punct	sits
7	She is reading a book.	She	she	PRON	nsubj	reading
8	She is reading a book.	is	be	AUX	aux	reading
9	She is reading a book.	reading	read	VERB	ROOT	reading
10	She is reading a book.	a	a	DET	det	book
11	She is reading a book.	book	book	NOUN	dobj	reading
12	She is reading a book.	.	.	PUNCT	punct	reading
13	John and Mary went to the market.	John	John	PROPN	nsubj	went
14	John and Mary went to the market.	and	and	CCONJ	cc	John
15	John and Mary went to the market.	Mary	Mary	PROPN	conj	John
16	John and Mary went to the market.	went	go	VERB	ROOT	went
17	John and Mary went to the market.	to	to	ADP	prep	went
18	John and Mary went to the market.	the	the	DET	det	market
19	John and Mary went to the market.	market	market	NOUN	pobj	to

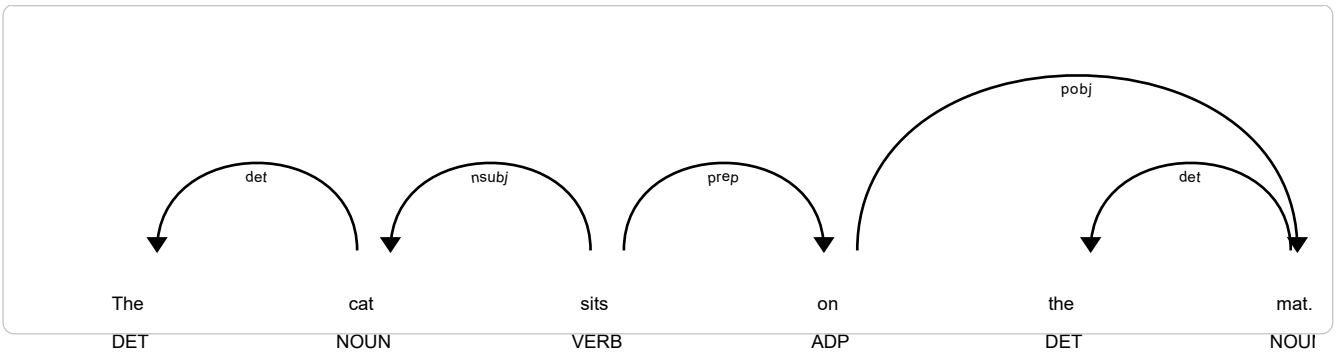
Next steps: Generate code with df New interactive sheet

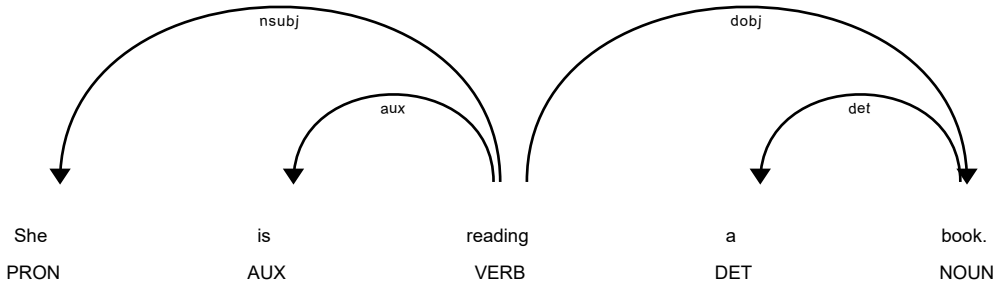
```
"""
STEP 5: Identify Sentence Structure

We extract:
- Subject (nsubj)
- Object (dobj / pobj)
- ROOT verb
- Modifiers (amod, advmod)

Dependency parsing helps identify relationships like:
who did what to whom.
"""

for doc in docs:
    print("\nSentence:", doc.text)

    subject = []
    obj = []
    root = ""
    modifiers = []

    for token in doc:
        if token.dep_ == "nsubj":
            subject.append(token.text)
        if token.dep_ in ["dobj", "pobj"]:
            obj.append(token.text)
        if token.dep_ == "ROOT":
            root = token.text
        if token.dep_ in ["amod", "advmod"]:
            modifiers.append(token.text)
```

```
    print("Subject:", subject)
    print("Verb (ROOT):", root)
    print("Object:", obj)
    print("Modifiers:", modifiers)
```

```
Sentence: The cat sits on the mat.
Subject: ['cat']
Verb (ROOT): sits
Object: ['mat']
Modifiers: []

Sentence: She is reading a book.
Subject: ['She']
Verb (ROOT): reading
Object: ['book']
Modifiers: []

Sentence: John and Mary went to the market.
Subject: ['John']
Verb (ROOT): went
Object: ['market']
Modifiers: []

Sentence: Although it was raining, we went outside.
Subject: ['it', 'we']
Verb (ROOT): went
Object: []
Modifiers: ['outside']

Sentence: He completed the assignment and submitted it.
Subject: ['He']
Verb (ROOT): completed
Object: ['assignment', 'it']
Modifiers: []

Sentence: The ball was thrown by the boy.
Subject: []
Verb (ROOT): thrown
Object: ['boy']
Modifiers: []

Sentence: Can you solve this problem?
Subject: ['you']
Verb (ROOT): solve
Object: ['problem']
Modifiers: []

Sentence: Artificial intelligence is transforming industries.
Subject: ['intelligence']
Verb (ROOT): transforming
Object: ['industries']
Modifiers: ['Artificial']

Sentence: The teacher explained the concept clearly.
Subject: ['teacher']
Verb (ROOT): explained
Object: ['concept']
Modifiers: ['clearly']

Sentence: After the meeting ended, everyone left quickly.
Subject: ['meeting', 'everyone']
Verb (ROOT): left
```

```
"""
STEP 6: Dependency Tree Visualization

displaCy is spaCy's built-in visualization tool.
It graphically represents dependency relationships between words.

This helps in understanding sentence structure visually.
"""

from spacy import displacy

# Visualize first 5 sentences
for i in range(5):
    doc = docs[i]
    displacy.render(doc, style="dep", jupyter=True)
```


pobj
det
nsubj
prep
det
The
cat
sits
on
the
mat.
DET
NOUN
VERB
ADP
DET
NOUI


nsubj
dobj
aux
det
She
is
reading
a
book.
PRON
AUX
VERB
DET
NOUN